Abstract Classes

It is sometimes the case that the parent class is too generalized to be of practical use in a project. This is quite normal and is actually a driver of good object-oriented design (making modules dependent on abstractions of other modules). With PHP it is possible to declare a parent class as `abstract`, along with some (or all) of its methods:

```
abstract class Car
{
    abstract public function getBrand(): string;

    abstract public function getModel(): string;
}

class VwGolf extends Car
{
    public function getBrand(): string
    {
        return 'Volkswagen';
    }

    public function getModel(): string
    {
        return 'Golf GTI';
    }
}
```

Here we have a parent class `Car` and a child class `VwGolf`. `Car` has been declared using the `abstract` keyword and has two abstract methods. Methods declared this way do not have an implementation; instead, any class extending the abstract parent is responsible for providing it. We can now use `Car` as a type declaration, and any child of `Car` will fulfil the dependency:

```
function carInfo(Car $car) {
    print "{$car->getBrand()} {$car->getModel()}" . PHP_EOL;
}
carInfo(new VwGolf());
```